Scelte Progettuali - Pipeline Telemetria Autobus

Autobus Smart

Simulazione di N autobus smart. Trattamento dell'autobus smart come oggetto, per cui uso di un linguaggio OOP, con attributi i dati da raccogliere per il sistema di telemetria, che saranno simulati.

Linguaggio di programmazione

La scelta ricade su un linguaggio OOP, dato che mi sembra abbastanza chiara la rappresentazione dell'autobus smart come un oggetto con tutti i suoi attributi attorno da gestire, questi ultimi saranno privati per information hiding e accessibili all'esterno solamente attraverso metodi specifici dell'oggetto.
Tra i linguaggi OOP trattati nel mio percorso sono presenti Java e Python che conosco maggiormente, e Python è sicuramente il linguaggio che a parer mio rende più diretta la connessione tra progettazione e realizzazione. Inoltre è sicuramente di più facile gestione, maggiormente flessibile e più carino. Mi ci trovo anche meglio a programmare.
Risulterebbe interessante uno sviluppo dell'applicazione in maniera concorrente, per cui utilizzare invece Java. Questo effettivamente aumenterebbe le prestazioni, consentirebbe di far eseguire un Thread per ogni autobus smart, ottenendo N di questi Threads, e quindi fare in modo che ognuno di questi sia indipendente. La motivazione che mi porta a considerare questa scelta non ottimale è l'assenza di uno stato condiviso tra i diversi Thread, perché le metriche all'interno di ogni oggetto autobus smart devono essere specifiche e non possono essere condivise, per cui è assente uno dei motivi che ha portato all'introduzione della programmazione concorrente.
Altro punto a favore di Python è legato ai virtual environment, con cui è molto facile avviare il venv e soprattutto installare le dipendenze del programma e rendere

portabili queste ultime, di conseguenza il programma stesso ne consegue una portabilità molto elevata.
Infine l'integrazione con il protocollo di comunicazione risulta essere molto meno verbosa e più semplice in Python rispetto a Java.
Scelta finale: **Python**.

Reference

https://www.ionos.it/digitalguide/siti-web/programmazione-del-sito-web/python-vs-java/

Sensoristica e metriche

Metriche

Metriche interessanti da raccogliere a partire dall'autobus smart:

- Posizione GPS;
- Velocità;
- Pressione degli pneumatici;
- Usura dei freni;
- Stato del motore, con monitoraggio del surriscaldamento dello stesso;
- Livello di carica delle batterie, livello di carburante;
- Temperatura batterie, problematica se anomala;
- Consumo di carburante ed emissioni;
- Numero di passeggeri a bordo;
- Dati ambientali: temperatura e umidità.
- Spia del motore, legata a stato del motore sotto una certa soglia o al surriscaldamento eccessivo;
- Panic Button, segnale di emergenza immediato alla centrale operativa, allarme associato alla posizione del mezzo.

Tra i dati associati all'autobus smart dovranno essere inseriti chiaramente anche un identificativo, ad esempio l'ID o il numero di serie del veicolo, e il timestamp per mantenere i dati ordinati temporalmente. Da notare che le ultime due metriche sono eventi, in quanto tali saranno gestiti poi in maniera differente nel sistema di Processing.

Sarà attuata una differenziazione dei dati a livello di motorizzazione degli autobus, per cui verrà sfruttata l'ereditarietà per costituire una classe Autobus generica avente alcune metriche generali al suo interno, e differenziare gli autobus sul tipo di motorizzazione con altre classi, per cui:

- Termica;
- Ibrida;
- Elettrica.

Questo per raccogliere dati che sono interessanti solamente nel contesto dell'autobus con una determinata motorizzazione. Per aiutare la differenziazione dei veicoli si userà un protocollo di comunicazione come MQTT che lavora a topic, per fare in modo che gli autobus con motorizzazione termica comunichino attraverso un topic di soli autobus termici, e così via per le altre motorizzazioni.
I dati presenti solamente in una certa motorizzazione sono:

- Livello di carica delle batterie: presente in autobus smart elettrici e ibridi.
- Livello di carburante: presente in autobus smart termici e ibridi.
- Temperatura batterie: presente in autobus smart elettrici e ibridi.
- Consumo carburante: presente in autobus smart termici e ibridi.

Formato invio metriche

Decisione fortemente dipendente dal protocollo di comunicazione scelto, per cui una possibile scelta sarebbe quella di usare CoAP con formato binario, per essere più lightweight e occupare meno banda possibile, però questo implica:

1. Necessità di comunicare via CoAP verso il sistema di Ingestion, che sarà costituito da Apache Kafka;
2. Necessità di convertire il messaggio in un formato differente, perché il formato binario non è human-readable ovviamente, quindi questo implicherebbe una forma di processing sul messaggio una volta comunicato.

La possibilità alternativa è quella del JSON, per essere maggiormente human-readable, in coppia al protocollo di comunicazione MQTT, che è lightweight allo stesso modo di CoAP, per cui il formato dati determina la dimensione del payload che sarà l'unica notevole nell'invio del messaggio.
Inoltre è di notevole importanza che la comunicazione avvenga Device-to-Cloud direttamente e non Device-to-Gateway-to-Cloud, quindi non essendoci un dispositivo

intermediario, e non essendoci il bisogno che i dispositivi debbano comunicare tra di loro, l'approccio a topic di MQTT in coppia ad un formato dati più human-readable è adottabile con vantaggi, perché il Device sarà limitato sì, ma comunque la rete su cui poi transita il messaggio non lo è affatto, dato che sarebbe costituita dalla rete cellulare wireless, che è tutt'altro che limitata a livello di banda.
Scelta finale: **JSON**.

Frequenza raccolta metriche

La frequenza di raccolta delle metriche può essere dimensionata, a parer mio, tra la volta al secondo e la volta ogni 5 secondi, perché tutti i dati raccolti non sono così critici, o meglio alcuni lo sono, ma difficilmente hanno un cambiamento così rapido tale per cui in 5 secondi implicano un failure grave dell'autobus smart.
La spia del motore e il panic button possono essere pensati come eventi, per cui inviati solamente in corrispondenza di specifiche occasioni.

References

- https://it.wikipedia.org/wiki/Automatic_vehicle_monitoring
- https://www.ibm.com/it-it/think/topics/telemetry
- https://www.buswhere.com/it/the-future-of-bus-tracking-innovations/
- https://salirporbarcelona.com/it/tmb-utilizza-un-sistema-di-telemetria-per-tracciare-i-suoi-autobus-a-barcellona/
- https://www.pluservice.net/it/soluzioni/monitoraggio-e-its-di-bordo

Sistema centralizzato

Posizionamento

Il case study analizzato è rappresentato dalla soluzione di monitoraggio adottata da Roma TPL, è una soluzione che opta per avere una centrale operativa in cui sono presenti server ridondati, 11 postazioni client e sistemi di deposito. La collocazione di queste risorse non viene resa nota, però viene evidenziata la partnership con Vodafone, per cui Roma TPL ,per la centrale operativa, deduco che abbia o tutte le risorse in loco per formare l'infrastruttura oppure, molto più probabilmente, le risorse

per formare l'infrastruttura sono fornite da Vodafone, seguendo un modello di Public o Community Cloud. Quindi in questa casistica è Vodafone attraverso i propri data center a fornire l'infrastruttura con un modello che almeno deve essere IaaS.
In tutti i restanti case study spesso viene evidenziata la presenza della centrale operativa o centro di controllo, però senza mai specificare come l'infrastruttura necessaria sia eretta.

In ogni caso se le risorse vengono acquistate dalla compagnia e quest'ultima decide, a seguito di determinate valutazioni, di ampliare la flotta, quasi sicuramente si renderà necessario un ampliamento delle risorse, ma questa soluzione adottata è poco scalabile per natura, perché è possibile solamente scalare verticalmente acquistando altre risorse computazionali. Al contrario se la soluzione viene posta in Cloud, con un minimo effort è possibile scalare orizzontalmente, senza avere problemi di configurazione manuale dei sistemi e tempi di attesa lunghi nell'acquisto delle risorse e nella loro configurazione.
Altro fattore da tenere in considerazione è quello rappresentato dalla privacy dei dati, per cui l'azienda di trasporti che gestisce la flotta di autobus smart potrebbe decidere di mantenere i dati in casa, per lo meno quelli più sensibili, optando quindi per posizionare il sistema centralizzato in una delle sedi della compagnia. Questo sicuramente può aiutare in termini di latenza, perché gli autobus smart dovrebbero inviare i dati ad un punto maggiormente vicino alla loro posizione, e inoltre può aiutare anche dal punto di vista della quantità dei dati da trattare, questione sicuramente importante dal momento che il sistema di telemetria lavora in real-time.
D'altra parte la scelta potrebbe essere controproducente perché la mole di dati da gestire è molta, e quindi inviare tutto ad un unico punto potrebbe risultare un collo di bottiglia se il traffico comincia a diventare di dimensioni rilevanti, inoltre introdurrebbe un single point of failure per il sistema centralizzato che sicuramente non è desiderato dalla compagnia.
Per cui una scelta migliore sotto questo punto di vista è quella di spostare tutto in Cloud, in modo da avere più nodi di elaborazione, e quindi rimuovendo il single point of failure. Inoltre non sarebbe nemmeno così sconveniente, lato invio dati dagli autobus smart, nel momento in cui si decide che tutti i dati vanno direttamente in Cloud, perché passando per la rete cellulare le informazioni non dovrebbero nemmeno transitare per una rete limitata, e andrebbero direttamente sulla rete

cellulare senza dover passare per la rete interna aziendale, che potrebbe portare maggiore overhead temporale. D'altra parte viene fuori anche un tema di sicurezza dei dati e anche di elaborazione totalmente in Cloud, per cui essendo il sistema real-time se ho bisogno di una responsiveness alta allora sarebbe meglio qualcosa di ibrido, per cui una soluzione Edge-Computing sarebbe molto probabilmente la migliore.
Questo perché riesce ad unire le esigenze di processamento a bassa latenza, dato che il sistema tratta i dati in real-time, e inoltre la possibilità di mantenere alcuni dati, quelli più sensibili, "in casa" mentre quelli meno sensibili processarli in Cloud.

In conclusione, una soluzione Cloud privata on-premise dipende fortemente dallo stato della compagnia, se questa è ben avviata e ha già in casa l'infrastruttura per gestire un applicativo del genere allora è conveniente, mentre nel momento in cui questa strumentazione non è disponibile allora la strada si fa tortuosa, anche per le difficoltà nel gestire una piattaforma Cloud privata con uno strumento come OpenStack ad esempio, che è molto complesso da gestire. Per cui dato che la compagnia tratta come main business i trasporti, è difficile prevedere che quest'ultima abbia apparecchiature per garantire un'infrastruttura Cloud privata, di conseguenza una soluzione pubblica si rende preferibile.
Scelta finale: **Edge-Computing**.

Processing

Per quanto concerne il processamento la soluzione ritenuta migliore è un processing in stream, dato che i dati provenienti dal sistema vanno analizzati in real-time, in particolare analizzandoli rispetto al processing-time.
Data la natura real-time del sistema un processing a batch non è realizzabile, si potrebbe pensare eventualmente ad un processamento micro-batch, però l'esigenza maggiore è la latenza, e soprattutto finché i dati raccolti sono in uno stato positivo è tutto ok ed è possibile anche processarli in micro-batch, però nel momento in cui si verifica qualcosa di grave il sistema è pensato per comunicarlo in maniera immediata, appunto in real-time, mentre se invece il dato è inserito nel micro-batch e

magari si trova all'inizio di questo, si rende necessaria un'attesa per conoscere quell'informazione.
Scelta finale: **stream-oriented**.

References

- https://www.smau.it/casi-di-successo/una-soluzione-di-monitoraggio-della-flotta-del-trasporto-locale-e-di-gestione-delle-aree-a-traffico-limitato-per-un-migliore-servizio-al-cittadino-il-caso-roma-tpl
- https://datacentremagazine.com/company-reports/how-vodafone-is-overhauling-its-data-centre-infrastructure
- https://mediamobileitalia.com/gestione-flotta-avm/
- https://www.leonardo.com/it/news-and-stories-detail/-/detail/leonardo-innovative-solutions-for-safe-and-sustainable-transport
- https://www.cronacacomune.it/media/uploads/allegati/44/scheda_dettagliata_sistema_informativo_tper.pdf
- https://www.pluservice.net/it/soluzioni/monitoraggio-e-its-di-bordo

Tecnologie

- Protocollo di comunicazione: **MQTT**, per il modello push-based che permette di inviare messaggi e dati ai topic e immediatamente i consumer iscritti ricevono il messaggio, quindi semplifica la comunicazione in un contesto real-time, e inoltre abbassa la latenza. Può essere messo a paragone con CoAP perché anch'esso è pensato per dispositivi o reti con capacità limitate, per cui fitta bene in un contesto IoT, che risulta essere constrained. Ha la proprietà di avere un header molto ristretto, questo permette di adottare un formato dati che può anche portare un maggiore overhead, dato che la dimensione del messaggio sarà quasi unicamente data dal payload.
- Ingestion: **Apache Kafka**, per il successo che ha avuto come sistema di messaggistica e come piattaforma di comunicazione a qualsiasi livello, Enterprise e non, per cui è la soluzione dominante.

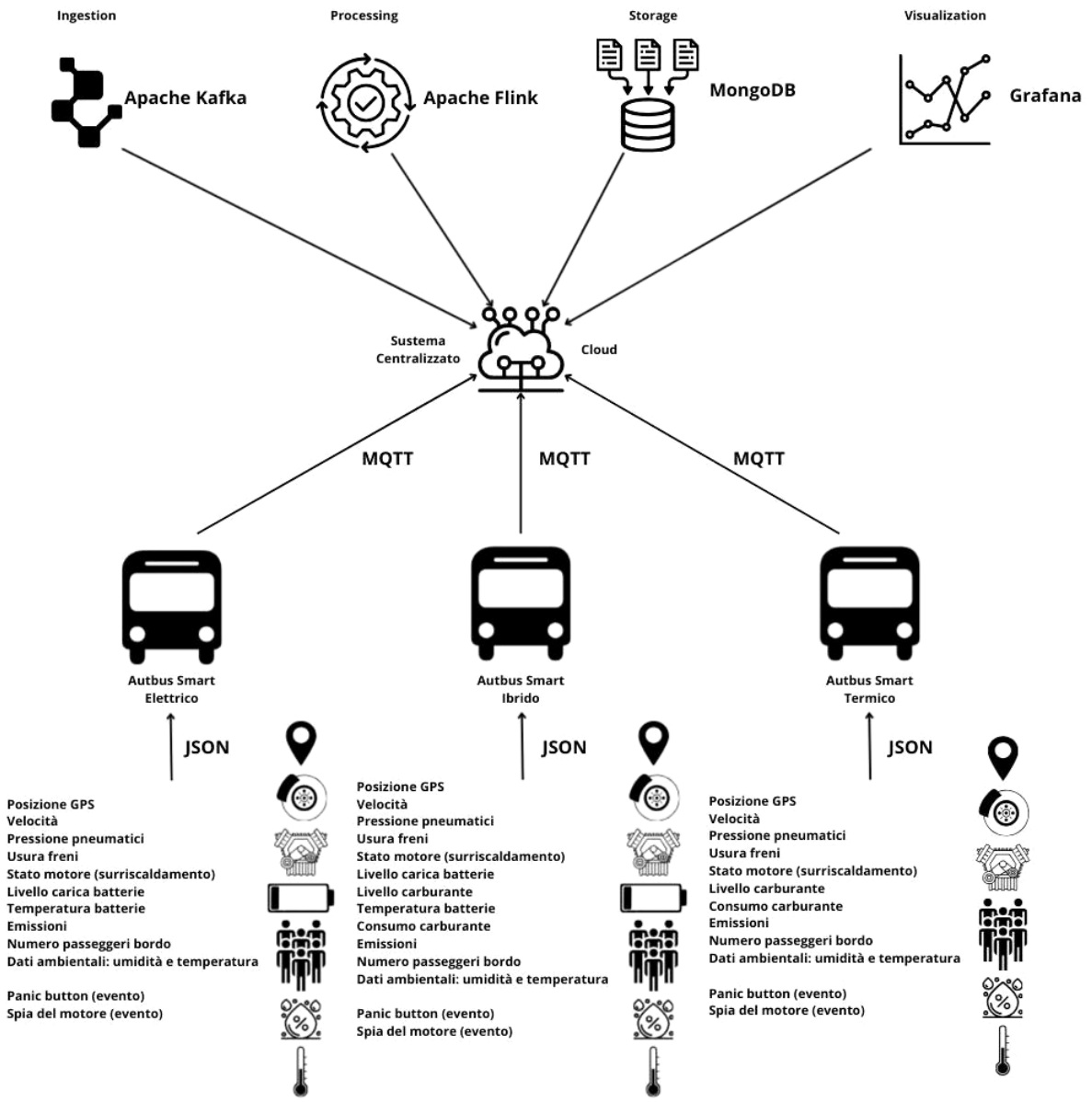
- Elaborazione / Processing: **Apache Flink**, uno degli strumenti dominanti all'interno del campo del processing stream-oriented, soluzione scelta a livello di architettura per il processamento in real-time dei dati.
- Storage: **MongoDB**, dato che la scelta del formato dati ricade sul JSON è molto semplice pensare ad un sistema di Storage non-relazionale che opera a documenti. MongoDB ne è la rappresentazione perfetta, utilizza un formato interno di documenti che risulta essere pressoché uguale al JSON se non per il nome, ossia BSON (Binary JSON), e di conseguenza il formato.
- Visualizzazione: **Grafana**, a livello di visualizzazione sicuramente uno degli strumenti più affermati, Kibana è l'altra possibilità ma è abbastanza simile e soprattutto maggiormente pensata per lo Stack Elastic, in cui lo strumento è maggiormente integrato con la tecnologia Elasticsearch.

References

- https://mqtt.pro/mqtt-vs-kafka/
- https://ankittrehan2000.medium.com/amqp-rabbitmq-vs-kafka-for-asynchronous-communication-4c4dd703819f
- https://hackernoon.com/the-streaming-bridges-a-kafka-rabbitmq-mqtt-and-coap-example-to-learn-more
- https://developers.redhat.com/articles/2022/01/19/connect-iot-devices-drogue-iot-and-openshift-streams-apache-kafka

Architettura Telemetria Autobus

Il flusso di comunicazione all'interno dell'architettura è rappresentato dagli autobus smart, con diverse motorizzazioni, che inviano le metriche raccolte verso il sistema di Ingestion, Apache Kafka, mediante il protocollo di comunicazione MQTT. Successivamente le informazioni passano dal sistema di Ingestion verso quello di Processing, ossia Apache Flink, poi, una volta elaborati, i dati transitano al sistema di Storage, rappresentato da MongoDB, e infine al sistema di Visualization, Grafana. Qui l'architettura opera storage dei dati solamente dopo averli processati, una possibile alternativa potrebbe essere quella di prevedere una memorizzazione anche dei raw data, con un'ottica Medallion Architecture.


Ingestion
Processing
Storage
Visualization
Apache Kafka
Apache Flink
MongoDB
Grafana
Sustema Centralizzato
Cloud
MQTT
MQTT
MQTT
Autbus Smart Elettrico
Autbus Smart Ibrido
Autbus Smart Termico
JSON
JSON
JSON
Posizione GPS
Velocità
Pressione pneumatici
Usura freni
Stato motore (surriscaldamento)
Livello carica batterie
Temperatura batterie
Emissioni
Numero passeggeri bordo
Dati ambientali: umidità e temperatura
Panic button (evento)
Spia del motore (evento)
Posizione GPS
Velocità
Pressione pneumatici
Usura freni
Stato motore (surriscaldamento)
Livello carica batterie
Livello carburante
Temperatura batterie
Consumo carburante
Emissioni
Numero passeggeri bordo
Dati ambientali: umidità e temperatura
Panic button (evento)
Spia del motore (evento)
Posizione GPS
Velocità
Pressione pneumatici
Usura freni
Stato motore (surriscaldamento)
Livello carburante
Consumo carburante
Emissioni
Numero passeggeri bordo
Dati ambientali: umidità e temperatura
Panic button (evento)
Spia del motore (evento)